

PRO121 Fundamentos de Programación

Ejercicios - Archivos

Ejercicio 1: Reporte diario de producción en minería

La empresa "Minerales del Futuro" recopila semanalmente reportes de extracción de minerales. Estos reportes se almacenan en un archivo de texto, con el objetivo de procesarlo para extraer los datos necesarios y generar un resumen de la producción.

El archivo de texto (**reporte_mineral.txt**, disponible en el Aula Virtual) contiene una serie de reportes, uno por línea. Cada línea sigue el siguiente formato: **tipo-cantidad**, donde:

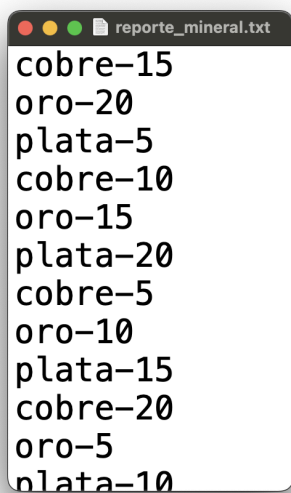
- **tipo**: una palabra que representa el tipo de mineral (sólo letras, sin espacios, sin símbolos o caracteres especiales: cobre, oro o plata; no existe otro mineral).
- **cantidad**: un número entero positivo que indica la cantidad de mineral extraída (en kilogramos).

Objetivo: Desarrollar un programa en Python que procese un archivo de texto que contiene los reportes de producción minera. El programa debe leer estos datos, extraer la información relevante y generar un resumen de la producción semanal.

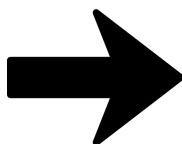
- **Leer el archivo**: leer el archivo para obtener la información minera.
- **Procesar lista de datos**: calcular la cantidad total de mineral extraído.
- **Generar un archivo resumen**: utilizando la información obtenida en el punto anterior, genera un archivo nuevo llamado **resumen_reporte_mineral.txt**, con el valor de cuánto cobre se extrajo, cuánto oro se extrajo y cuánta plata se extrajo.

Ejemplo:

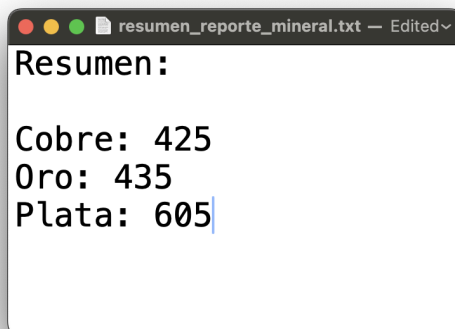
Archivo fuente con datos
reporte_mineral.txt



```
cobre-15
oro-20
plata-5
cobre-10
oro-15
plata-20
cobre-5
oro-10
plata-15
cobre-20
oro-5
plata-10
```



Archivo salida con el resumen
resumen_reporte_mineral.txt



```
Resumen:

Cobre: 425
Oro: 435
Plata: 605
```

Ejercicio 2: Registro de pagos

La empresa "Minerales del Futuro", convencida de su potencial, le ha solicitado desarrollar un sistema de pago de sueldos para sus empleados. Cada empleado tiene un salario base líquido mensual (fuera ya de impuestos), pero éste puede variar debido a las horas extras trabajadas. La información sobre los salarios y horas extras, se almacena en un archivo de texto, que contiene el ID del empleado, su salario base, y las horas extras trabajadas. El valor de una hora extra, equivale al 5% del sueldo líquido.

El archivo de texto (**reporte_sueldo.txt**, disponible en el Aula Virtual) contiene una serie de registros, uno por línea. Cada línea sigue el siguiente formato: **ID_empleado-salario_base-horas_extras**, donde:

- **ID_empleado**: identificador único del empleado.
- **salario_base**: el salario base mensual del empleado (número entero).
- **horas_extras**: la cantidad de horas extras trabajadas (número entero).

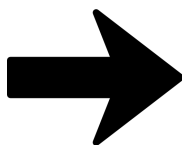
Objetivo: Desarrollar un programa en Python que procese un archivo de texto con los datos de sueldos de los empleados, calcule el nuevo sueldo líquido de cada empleado (si realiza horas extras).

- **Leer el archivo**: leer el archivo para obtener la información de salarios.
- **Procesar lista de datos**: calcular el nuevo sueldo, si y sólo si, el empleado realiza horas extras; si no realiza, mantiene el sueldo anterior.
- **Generar un nuevo archivo**: utilizando la información obtenida en el punto anterior, genera un archivo nuevo llamado **reporte_sueldo_actualizado.txt**.

Ejemplo:

Archivo fuente con datos
reporte_sueldo.txt

```
reporte_sueldos.txt
ID1001-1501-9
ID1002-1584-6
ID1003-1695-0
ID1004-1511-5
ID1005-1623-0
ID1006-1916-10
ID1007-1797-0
ID1008-1981-0
ID1009-1679-4
ID1010-1892-6
ID1011-1592-8
ID1012-1911-3
ID1013-1540-3
```



Archivo salida con el resumen
reporte_sueldo_actualizado.txt

```
reporte_sueldo_actualizado.txt
ID1001-2176.45
ID1002-2059.2
ID1003-1695
ID1004-1888.75
ID1005-1623
ID1006-2874.0
ID1007-1797
ID1008-1981
ID1009-2014.8
ID1010-2459.6
ID1011-2228.8
ID1012-2197.65
ID1013-1771.0
```

PRO121 Fundamentos de Programación

Ejercicios - Archivos

Ejercicio 3: Inventario de medicamentos

Una nueva cadena de farmacia desea implementar un sistema para gestionar su inventario de medicamentos. La información sobre cada medicamento se encuentra almacenada en un archivo CSV, que incluye detalles como el nombre del medicamento, su cantidad disponible, su vigencia, y su categoría (por ejemplo, analgésico, antibiótico, antiinflamatorio, etc.). La empresa le ha solicitado que procese ese archivo, para actualizar aquellos medicamentos que ya no tienen stock (no quedan productos disponibles) y aquellos que ya vencieron.

El archivo de texto (`inventario_medicamentos.csv`, disponible en el Aula Virtual) contiene una serie de registros, uno por línea. Cada línea sigue el siguiente formato: **nombre,cantidad,vigente,categoría**, donde:

- **nombre**: el nombre del medicamento.
- **cantidad**: la cantidad disponible en stock (número entero).
- **vigente**: indica si el medicamento está vigente o no (**True** o **False**).
- **categoría**: La clasificación del medicamento (analgésico, antibiótico, antiinflamatorio, etc.).

Objetivo: Desarrollar un programa en Python que procese un archivo de texto CSV con los datos de medicamentos y genere una versión actualizada.

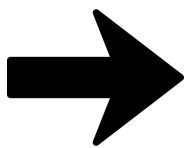
- **Leer el archivo**: leer el archivo para obtener la información de salarios.
- **Procesar lista de datos**: eliminar todo medicamento vencido y todo medicamento que no tenga stock.
- **Generar un nuevo archivo**: utilizando la información obtenida en el punto anterior, genera un archivo nuevo llamado `inventario_medicamentos_actualizado.csv`.

Ejemplo:

Archivo fuente con datos

`inventario_medicamentos.csv`

```
Nombre,Cantidad,Disponible,Categoría
Paracetamol,3,False,Antibiotico
Ibuprofeno,3,False,Analgesico
Amoxicilina,3,True,Antihipertensivo
Aspirina,2,False,Antiinflamatorio
Diclofenaco,3,True,Antibiotico
Metformina,2,True,Analgesico
Losartan,3,True,Antiinflamatorio
Omeprazol,5,True,Analgesico
Clonazepam,0,True,Antibiotico
Captopril,4,False,Antihipertensivo
Simvastatina,2,False,Antihipertensivo
Azitromicina,2,True,Analgesico
Enalapril,5,True,Antihipertensivo
Levotiroxina,1,True,Antibiotico
Furosemida,5,False,Antibiotico
```



Archivo salida con el resumen

`inventario_medicamentos_actualizado.csv`

```
Nombre,Cantidad,Disponible,Categoría
Amoxicilina,3,True,Antihipertensivo
Diclofenaco,3,True,Antibiotico
Metformina,2,True,Analgesico
Losartan,3,True,Antiinflamatorio
Omeprazol,5,True,Analgesico
Azitromicina,2,True,Analgesico
Enalapril,5,True,Antihipertensivo
Levotiroxina,1,True,Antibiotico
Loratadina,1,True,Analgesico
Prednisona,5,True,Antihipertensivo
Insulina,3,True,Analgesico
Amiodarona,1,True,Antiinflamatorio
Bromazepam,4,True,Analgesico
Atenolol,3,True,Antihipertensivo
Esomeprazol,5,True,Analgesico
```

PRO121 Fundamentos de Programación

Ejercicios - Archivos

Ejercicio 4: Transferencias bancarias

Un banco desea implementar un sistema para registrar y analizar las transferencias bancarias realizadas por sus clientes. Cada transferencia incluye información como el número de cuenta de origen, el número de cuenta de destino, el monto transferido, y la fecha de la transferencia. Los datos de las transferencias se almacenan en un archivo CSV.

El archivo de texto (**transferencias_bancarias.csv**, disponible en el Aula Virtual) contiene una serie de registros, uno por línea. Cada línea sigue el siguiente formato: **cuenta_origen,cuenta_destino,monto,fecha**, donde:

- **cuenta_origen**: el número de cuenta que realiza la transferencia.
- **cuenta_destino**: el número de cuenta que recibe la transferencia.
- **monto**: monto transferido (entero positivo).
- **fecha**: la fecha de la transferencia (formato **DD-MM-AAAA**).

Objetivo: Desarrollar un programa en Python que procese un archivo CSV con las transferencias bancarias, permita verificar si hay transferencias por encima de un monto específico.

- **Leer el archivo:** leer el archivo para obtener la información de salarios.
- **Procesar lista de datos:** revisar las transferencias que superen un monto ingresado por el usuario (validar); registrar la cuenta de origen y el monto transferido.
- **Generar un nuevo archivo:** utilizando la información obtenida en el punto anterior, genera un archivo nuevo llamado **transferencias_bancarias_premium.csv**.

Ejemplo:

Archivo fuente con datos

transferencias_bancarias.csv

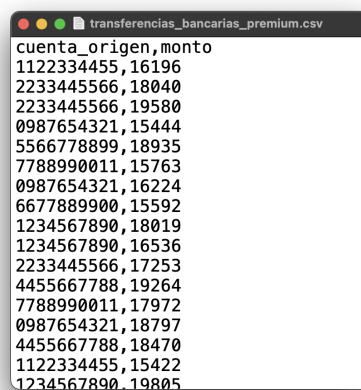
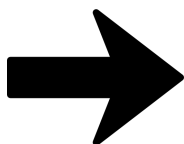
Archivo salida con el resumen

transferencias_bancarias_premium.csv

(Ingrese un monto: 15000)



```
cuenta_origen,cuenta_destino,monto,fecha
2233445566,5566778899,2486,14-10-2024
2233445566,1122334455,6207,19-10-2024
1122334455,0987654321,13245,29-04-2024
4455667788,2233445566,7057,19-12-2024
6677889900,7788990011,4136,13-06-2024
2233445566,5566778899,1344,17-06-2024
6677889900,1122334455,5263,26-07-2024
1122334455,0987654321,16196,08-05-2024
5566778899,1122334455,5445,08-05-2024
2233445566,5566778899,18040,12-09-2024
6677889900,2233445566,5877,30-06-2024
7788990011,2233445566,13051,07-05-2024
2233445566,6677889900,19580,08-05-2024
5566778899,0987654321,13883,21-08-2024
4455667788,6677889900,2393,05-07-2024
7788990011,2233445566,10985,05-11-2024
2233445566,1122334455,5382,30-10-2024
```



```
cuenta_origen,monto
1122334455,16196
2233445566,18040
2233445566,19580
0987654321,15444
5566778899,18935
7788990011,15763
0987654321,16224
6677889900,15592
1234567890,18019
1234567890,16536
2233445566,17253
4455667788,19264
7788990011,17972
0987654321,18797
4455667788,18470
1122334455,15422
1234567890,19805
```

BONUS: ¿Qué día se hizo la mayor transferencia? ¿Cuál fue la cuenta que recibió la menor transferencia?

Salida en la terminal

Ingrese un monto: 15000

La mayor transferencia fue de 19963 y realizada el 21-11-2024

La menor transferencia fue de 1029 y realizada a 7788990011